

Création et analyse améliorée de la base de données de Prime video

Base de données – Project Final

avec

Prof. Selena Baset

21 mai 2025



Auteurs et numéro d'immatriculation:

CHOUIKHA Samar / 22-827-034

KPETO Ilem Nancy / 24-508-863

NIAMKE Marie Esther / 24-507-097

NIAMKE Sephora Emmanuela / 22-502-132

AYED Mohamed Aziz / 22-282-667

Table des matières

1. Introduction Contexte Métier	3
2.Diagramme de classe	4
3.Modèle Logique de Données (Modèle relationnel)	6
3.1 Versions antérieures et commentaires	6
4. Scripts de Création et d'Alimentation de la Base de Données	8
5. Requêtes SQL (Annexe A)	9
5.1. Requête 1: Top des genres des oeuvres les plus regardés par pays	9
5.2. Requête 2: nombre de visionnages par mois	9
5.3. Requête 3: Recommandation d'œuvres basée sur les favoris et les langues doublées	9
5.4. Requête 4: œuvres dont la saison disparaîtra bientôt	10
5.5. Requête 5: affichage du prix d'abonnement en devise locale	10
5.6. Requête 6: trigger - vérification d'abonnement actif	10
5.7. Conclusion de l'écriture des requêtes SQL	11
6. Présentation des visualisations	11
7. Utilisation d'outil d'IA	13
8. Bonus	13
9. Répartition du travail	14
10. Conclusion	15

1. Introduction Contexte Métier

Pour notre étude, on a choisi comme sujet Prime vidéo, une plateforme de streaming vidéo appartenant à Amazon. Elle permet d'accéder à une large gamme d'éléments formats vidéo tels que : films, séries, animés japonais.... Nous avons estimé qu'il était pertinent de nous intéresser à cette plateforme streaming, car nous y avons observé plusieurs limites qui peuvent freiner l'expérience utilisateur¹.

L'un des problèmes majeurs concerne le système de recommandations. Contrairement à d'autres plateformes comme Netflix, celui de Prime Vidéo semble moins avancé. Il ne s'appuie pas suffisamment sur les préférences ou les comportements passés des utilisateurs, ce qui rend les suggestions peu pertinentes et parfois déconnectées des goûts réels des abonnés à la plateforme.

Pour mieux étudier le contexte et ce qu'on peut améliorer sur la plateforme, nous avons aussi cherché à comprendre les opérations clés sur cette dernière telles que :

- La gestion des comptes utilisateurs et des abonnements.
- Le stockage et l'organisation du catalogue de vidéos.
- Le suivi de l'historique de visionnage.
- La recommandation de contenus personnalisés.
- Le classement des vidéos par genre, langue, durée, popularité, etc.

Toutes ces opérations ont besoin d'une base de données bien structurée permettant une récupération efficace des données afin d'assurer leur utilisation pour les besoins de Prime Vidéo. C'est cette solution que nous aborderons principalement dans ce projet.

Notre solution va s'adresser à différents acteurs pour lesquels notre étude pourrait sembler pertinente: les utilisateurs qui visionnent les vidéos et interagissent avec la plateforme et les équipes Amazon Prime qui gèrent le contenu et le fonctionnement technique.

Une base de données bien structurée permettrait effectivement de résoudre ces problèmes². Elle sera utile pour :

- Mieux connaître les préférences utilisateurs.
- Améliorer les suggestions personnalisées
- Organiser plus efficacement les vidéos selon plusieurs critères pertinents

2. Diagramme de classe

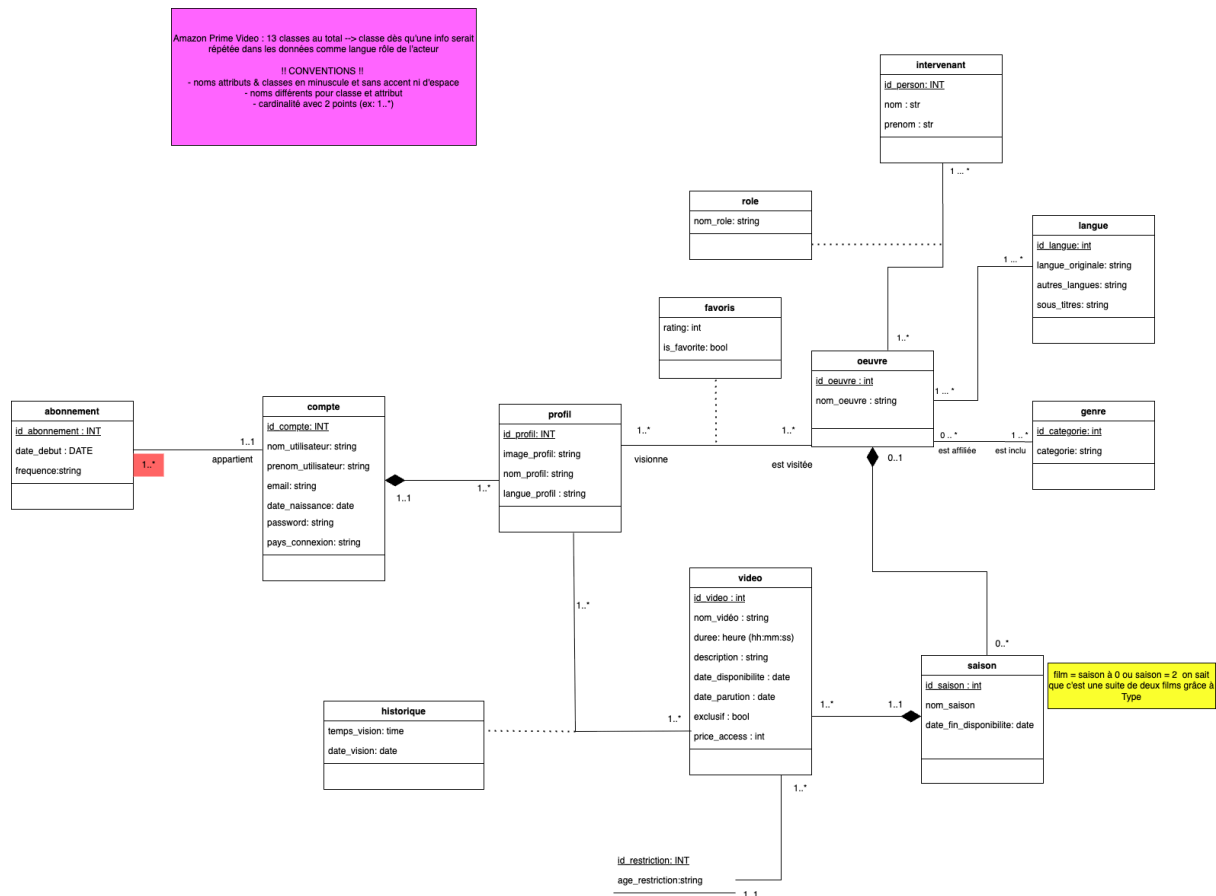
2.1 Version Antérieurs et commentaires

Pour nous donner une idée des classes à créer pour une base de données de plateforme de Streaming, nous avons fait des recherches sur la réalisation d'une telle base de données³ et avons créé un dictionnaire de données (Voir Annexe B) avant de réaliser le diagramme de classe.

¹ Amazone <https://www.aboutamazon.fr/actualites/divertissement/tout-savoir-sur-amazon-prime-video>

² Medium <https://www.geeksforgeeks.org/how-to-design-a-database-for-amazon-prime/>

³ geeksforgeeks <https://www.geeksforgeeks.org/how-to-design-a-database-for-amazon-prime/>



Dans notre **premier diagramme**, nous avons déjà intégré les **classes principales** nécessaires à la modélisation d'une plateforme de streaming, telles que abonnement, compte, profil, oeuvre, rôle, ainsi que les entités associées comme intervenant, langue, genre, vidéo, saison, historique et restriction.

Cependant, plusieurs **limites** étaient présentes :

- La classe vidéo était **peu exploitée** et n'avait pas tous ses attributs bien définis et pertinents.
- Certaines **classes manquaient de précision**, ce qui rendait la base de données incomplète.
- Le **lien entre abonnement, compte et profil restait flou**, notamment au niveau des cardinalités et des relations logiques.
- La classe langue regroupait trop d'informations à la fois (langue originale, doublée, sous-titres), ce qui nuisait à la **normalisation**.
- Le champ exclusif n'était pas isolé dans une entité bien définie, et causait une redondance des valeurs (Vrai et Faux).

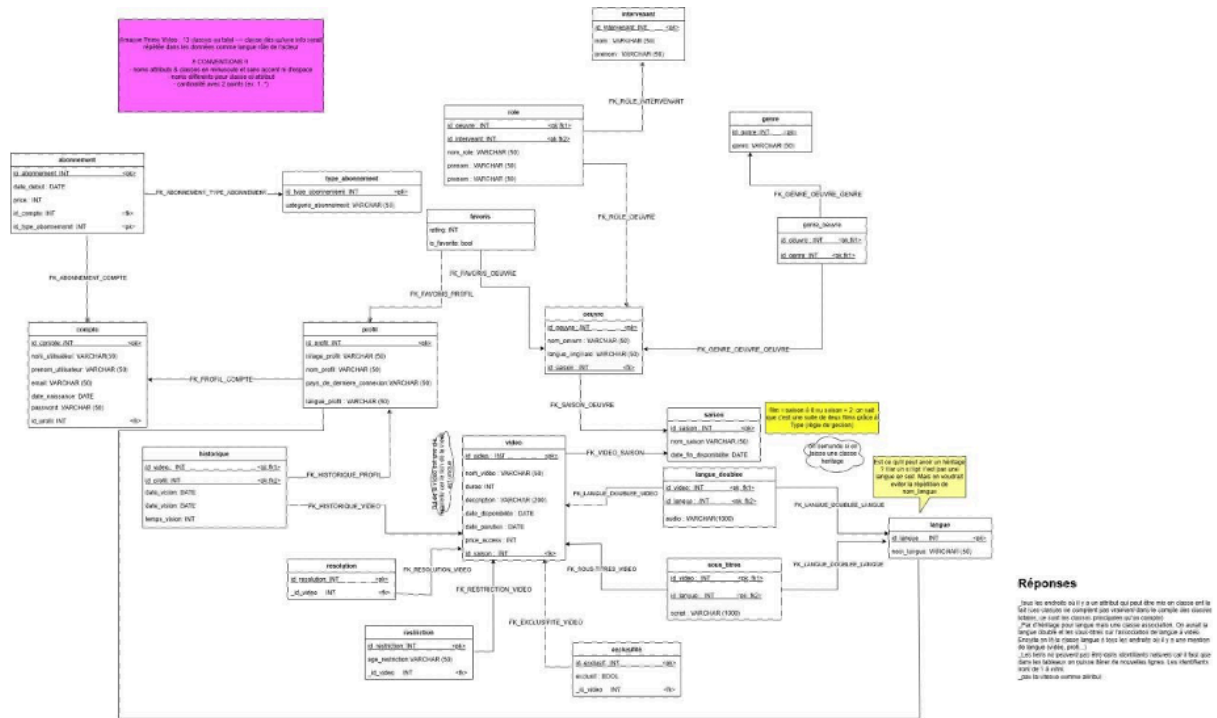
2.2 Version finale et commentaires

contenus similaires ou leur montrer la **popularité** du film via des **statistiques de visionnage**.

- Enfin, la classe exclusivité permet de **distinguer les contenus exclusifs à Amazon Prime**, une fonctionnalité importante pour tout service de streaming moderne.

3.Modèle Logique de Données (Modèle relationnel)

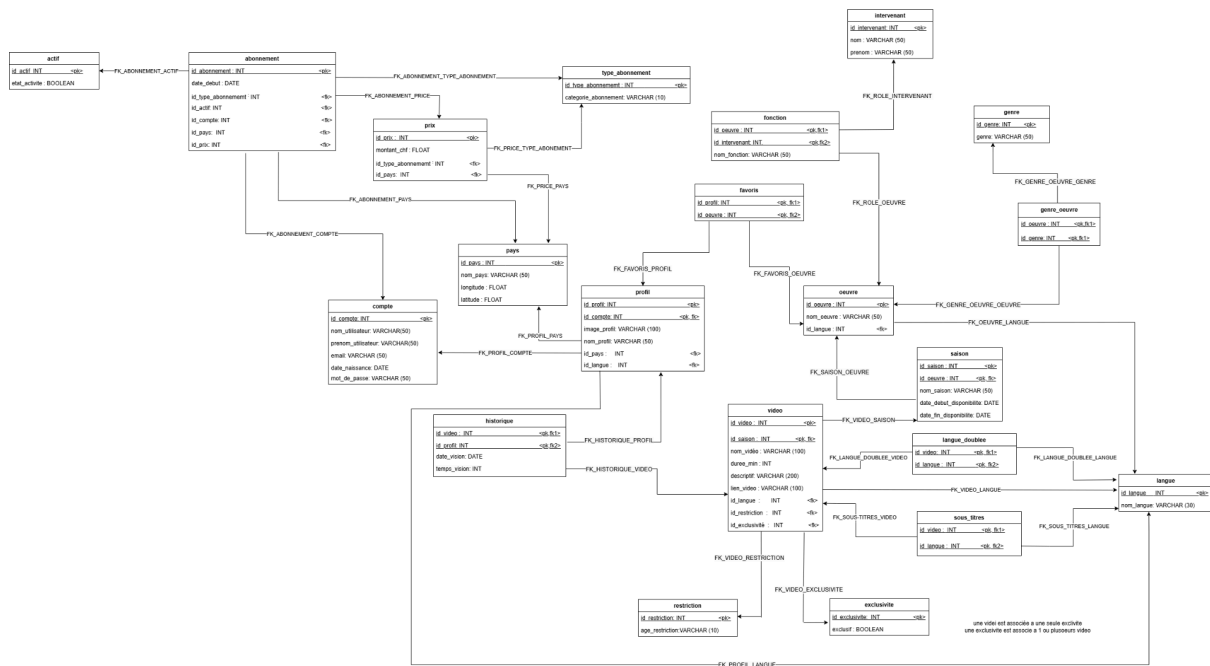
3.1 Versions antérieures et commentaires



Dans cette **première version du modèle relationnel**, nous avons traduit les principales entités de notre diagramme de classes en tables relationnelles, en respectant les conventions de nommage SQL les tables comme compte, profil, abonnement, vidéo ou saison sont bien structurées, avec des clés primaires et étrangères explicites.

La première version du diagramme relationnel présente une base fonctionnelle mais reste limitée sur plusieurs aspects. Les relations entre vidéo, langue, langue doublée et sous-titres sont floues, ce qui nuit à la normalisation du modèle. De plus, le lien entre abonnement et compte manque de clarté. La gestion des langues est regroupée dans une seule entité, ce qui complique l'ajout de fonctionnalités multilingues. Enfin, le champ exclusif n'est pas isolé dans une entité distincte, ce qui nuit à la modularité on peut pas l'utiliser comme on veut ni le réutiliser.

3.2 Version finale et commentaires



La **version finale** du modèle relationnel répond de manière cohérente aux limites identifiées dans le modèle précédent. Elle adopte une approche plus normalisée en s'inspirant des bonnes pratiques de modélisation relationnelle et des contraintes métier spécifiques à une plateforme de streaming.

Les **éléments linguistiques** ont été entièrement repensés :

- La table **LANGUE** centralise les langues disponibles,
- La table **LANGUE_DOUBLÉE** relie chaque vidéo à ses versions doublées avec précision,
- La table **SOUS_TITRES** permet une gestion propre des scripts en plusieurs langues.

Ce découpage favorise une **normalisation en 3NF** et améliore la gestion multilingue du catalogue en gardant une clarté qui nous sera très utile pour la gestion de notre base de données.

Des **entités techniques** essentielles ont été ajoutées :

- **RESTRICTION**, pour modéliser les limites d'âge,
 - **EXCLUSIVITÉ**, pour marquer les contenus réservés à la plateforme.
- Ces ajouts rendent le modèle **plus proche de la réalité métier** et facilitent l'implémentation de filtres ou de règles d'affichage.

La table VIDÉO a également été enrichie avec un champ LIEN_VIDEO permettant d'associer chaque enregistrement à son fichier source. Cela répond à une **exigence technique essentielle** dans un contexte de streaming.

Le système d'abonnement a gagné en structure, avec l'introduction de TYPE_ABONNEMENT et ACTIF, qui permettent de suivre la **catégorie d'abonnement** et son **état de validité**. Les relations ABONNEMENT → COMPTE → PROFIL sont désormais **clairement modélisées** à l'aide de clés étrangères nommées, ce qui évite toute ambiguïté. La classe ROLE fut remplacée par FONCTION car "role" était un mot réservé dans SQL.

Enfin, les entités HISTORIQUE et FAVORIS sont bien intégrées, et permettent de **suivre le comportement des utilisateurs, d'enrichir l'expérience personnalisée, et d'alimenter des systèmes de recommandation** plus pertinents selon le goût

4. Scripts de Création et d'Alimentation de la Base de Données

Pour la mise en place de notre base de données, nous avons adopté une approche en plusieurs étapes, combinant génération automatique, complétion manuelle et importation via phpMyAdmin.

1. Génération des données de base

Afin de gagner du temps et de structurer les données de manière cohérente, nous avons utilisé l'outil **ChatGPT** pour la majorité des données afin de générer des jeux de données réalistes correspondant à nos différentes tables (profils, comptes, vidéos, œuvres, langues, etc.). Cela nous a permis d'obtenir rapidement un volume initial de données adaptées à notre modèle relationnel. Certains données comment le prix des abonnements ont été créer en s'inspirant d'autres données existantes⁴.

2. Complétion et ajustement manuel

Certaines tables, notamment les tables associatives ou techniques (par exemple langue_doublée, sous_titres, historique, favoris, profils, pays), ont été **complétées à la main pour les clés primaires et étrangères. Par exemple, pour la longitude latitude pour pouvoir utiliser dans tableau pour la carte on chercher sur internet les coordonnées de chaque pays et on a trouvé** . Cela nous a permis d'ajuster les valeurs, d'ajouter des cas particuliers, et de mieux représenter les relations complexes entre entités.

3. Organisation dans des fichiers Excel

L'ensemble des données générées ou modifiées a été structuré dans **des fichiers Excel**, avec une feuille par table, en veillant à bien respecter l'ordre des colonnes, les types de données, et les relations logiques entre tables (clés primaires et étrangères).

4. Importation dans phpMyAdmin

Les fichiers Excel ont ensuite été **convertis en fichiers CSV** puis importés via **phpMyAdmin**, à l'aide de l'option d'importation intégrée.

⁴ Phonandroid. (2023, avril 14). *Prix des abonnements Netflix dans 40 pays du monde*.
<https://www.phonandroid.com/prix-des-abonnements-netflix-dans-40-pays-du-monde.html>

Une fois les données importées, nous avons utilisé phpMyAdmin pour **générer automatiquement les scripts SQL d'insertion** (INSERT INTO ... VALUES (...)) ainsi que les scripts de création de tables.

5. Requêtes SQL (Annexe A)

5.1. **Requête 1: Top des genres des oeuvres les plus regardés par pays**

Cette requête identifie les genres les plus populaires dans chaque pays en comptant le nombre de visionnages par genre. Elle regroupe les résultats par pays et présente les genres les plus regardés par ordre décroissant de popularité.

Comment elle répond à la problématique :

- Permet d'identifier les préférences par région géographique
- Aide à personnaliser les recommandations selon la localisation de l'utilisateur
- Facilite l'organisation du catalogue avec une approche adaptée à chaque marché

5.2. **Requête 2: nombre de visionnages par mois**

Cette requête analyse les tendances de visionnage sur toutes les années en comptabilisant le nombre de visionnages par mois et identifie les œuvres les plus populaires chaque mois.

Comment elle répond à la problématique :

- Permet d'identifier les périodes de forte consommation de contenu
- Aide à comprendre les tendances saisonnières de visionnage
- Permet d'ajuster les recommandations en fonction de la période de l'année
- Facilite la planification stratégique des mises en avant de contenu

5.3. **Requête 3: Recommandation d'œuvres basée sur les favoris et les langues doublées**

Cette requête établit un système de recommandation basé sur deux critères:

1. La popularité des œuvres (nombre de favoris)
2. L'accessibilité linguistique (nombre de langues disponibles en doublage)

Le premier indicateur est le nombre de favoris attribués à chaque œuvre, qui représente une mesure directe de l'appréciation par notre communauté d'utilisateurs. Ce critère nous permet d'identifier les contenus qui suscitent un intérêt explicite auprès de notre audience.

Le second indicateur, plus original, exploite le nombre de langues dans lesquelles une œuvre est doublée. Nous partons du principe que plus une série ou un film est disponible en différentes langues, plus cette œuvre a démontré sa valeur ce qui justifié un investissement financier substantiel. En effet, le processus de doublage représente un coût significatif que

les producteurs et distributeurs n'engagent que pour des contenus ayant déjà prouvé leur potentiel commercial ou leur qualité exceptionnelle.

Comment elle répond à la problématique :

- Cette double approche présente plusieurs avantages majeurs : elle privilégie les contenus véritablement appréciés par les utilisateurs, favorise l'accessibilité linguistique, et met en avant des œuvres ayant déjà démontré leur valeur sur le marché international. De plus, elle permet de contourner les limitations des systèmes de recommandation algorithmiques complexes tout en offrant des résultats hautement pertinents.
- En implémentant cette requête dans notre base de données, nous pouvons désormais proposer à nos utilisateurs un catalogue organisé selon des critères de popularité réelle et d'accessibilité, améliorant considérablement leur expérience de navigation et de découverte de nouveaux contenus.

5.4. Requête 4: Œuvres dont la saison disparaîtra bientôt

Cette requête identifie les saisons d'œuvre qui vont être supprimées dans 2 ans à partir du 1er mai 2025 et calcule le nombre de jours restants avant leur expiration.

Comment elle répond à la problématique :

- Permet d'avertir les utilisateurs que certains contenus vont bientôt disparaître
- Évite de recommander des contenus qui ne seront plus disponibles à court terme
- Améliore la transparence concernant la disponibilité des contenus
- Permet de voir les performances actuelles des oeuvres qui sont amenés à bientôt disparaître pour prendre une décision sur le renouvellement de leur contrat

5.5. Requête 5: Affichage du prix d'abonnement en devise locale

Cette requête convertit les prix d'abonnement de base (en CHF) dans la devise locale de chaque pays, en appliquant des taux de conversion spécifiques pour chaque territoire.

Comment elle répond à la problématique :

- Améliore la transparence sur les coûts réels pour l'utilisateur
- Facilite la compréhension des offres dans le contexte local de l'utilisateur en affichant le pris dans sa monnaie
- Contribue à une meilleure expérience utilisateur en présentant les informations pertinentes dans un format familier

5.6. Requête 6: Trigger - vérification d'abonnement actif

Ce que fait la requête : Ce trigger empêche un même compte d'avoir plusieurs abonnements actifs simultanément en vérifiant si un abonnement actif existe déjà avant d'en ajouter un nouveau.

Comment elle répond à la problématique :

- Assure la cohérence de la base de données concernant les abonnements
- Empêcher le renouvellement d'un abonnement temps qu'il y en a un actif
- Permet une gestion plus claire des comptes en activité en les associant à un seul abonnement actif

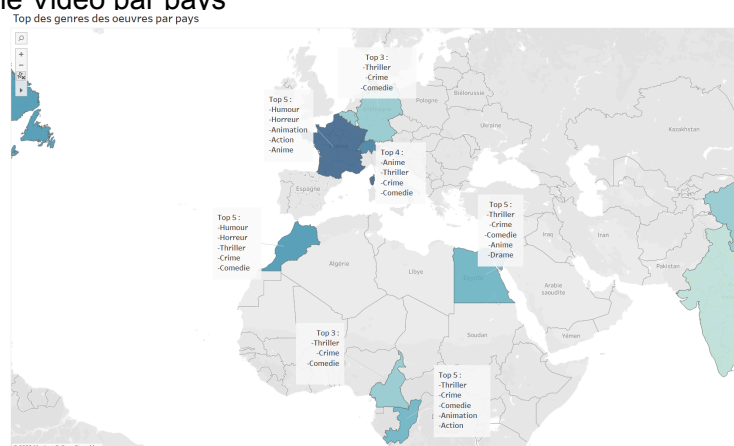
5.7. Conclusion de l'écriture des requêtes SQL

A travers un ensemble cohérent de requêtes SQL nous répondons précisément aux défis majeurs identifiés dans l'expérience Prime Vidéo. Grâce à l'analyse des préférences par région géographique et aux tendances temporelles de visionnage, nous atteignons une personnalisation significativement améliorée des contenus proposés. La pertinence des recommandations est considérablement renforcée par l'exploitation des données réelles de comportement utilisateur, notamment les favoris et l'historique de visionnage. Notre système introduit également une organisation plus intuitive du catalogue, enrichie par des informations transparentes sur la disponibilité temporelle des contenus, évitant ainsi les frustrations liées aux œuvres prochainement indisponibles. L'expérience utilisateur se trouve optimisée par l'intelligente priorisation des contenus accessibles dans la langue de l'utilisateur et par la clarté des informations tarifaires adaptées au contexte local. Cette convergence de fonctionnalités crée une plateforme où les recommandations deviennent véritablement pertinentes et où la navigation dans le catalogue devient fluide et intuitive, transformant fondamentalement la manière dont les utilisateurs découvrent et consomment les contenus sur Prime Vidéo.

6. Présentation des visualisations

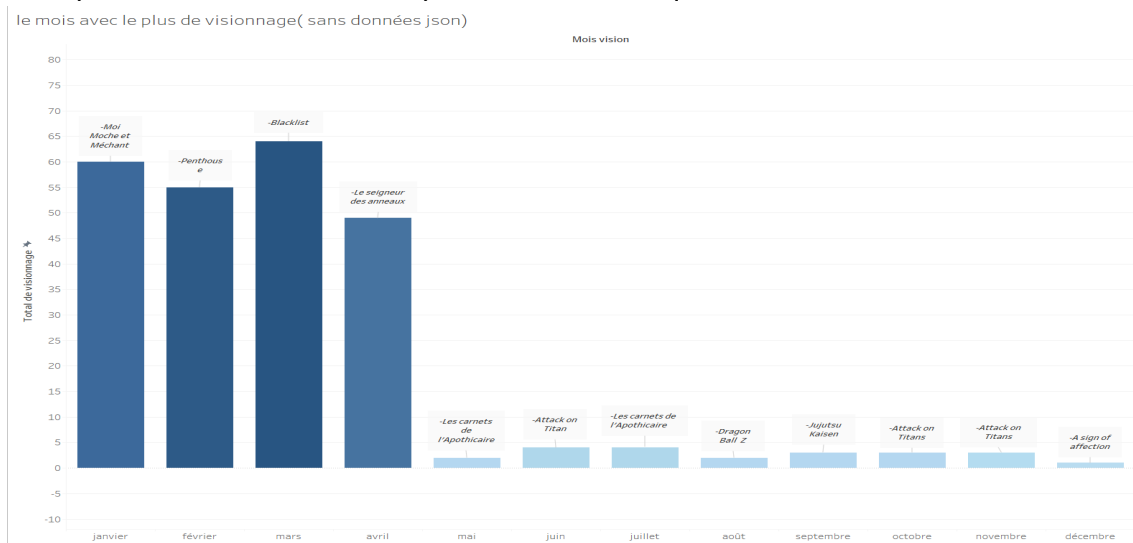
Nos trois visualisations sont basées partiellement sur le résultat des requêtes 1, 2 et 4.

Graphique 1 : Cartographie du top des genres par pays et de la répartition des profils d'utilisateur de Prime Vidéo par pays



En examinant l'intensité des couleurs – qui reflète le nombre de profils par pays – on constate que le Top des genres les plus consommés varie selon la région. Par exemple, en France, ce sont les œuvres humoristiques qui arrivent en tête, alors qu'au Congo c'est l'animation qui domine. Ces renseignements pourraient aider l'équipe de Prime Vidéo à recommander à ses abonnés des films et séries en fonction des préférences de chaque pays.

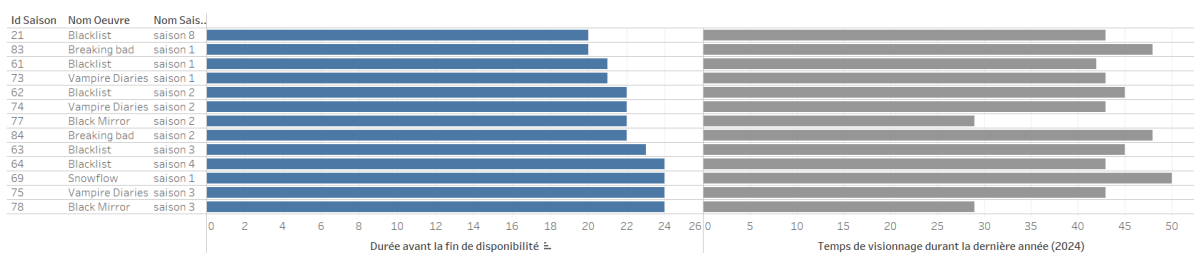
Graphique 2 : Diagramme en barre verticale du nombre de visionnage de vidéos par mois avec la présentation de l'œuvre la plus visionnée chaque mois.



Notre visualisation révèle que Mars est le mois avec le pic de visionnage, avec BlackList qui est l'œuvre la plus visionnée sur ce mois. D'autres mois comme Janvier et Février totalise beaucoup de visionnage.

Cette information cruciale permet à Prime Vidéo d'identifier précisément les périodes de fort engagement et de pouvoir optimiser ses serveurs pour accueillir l'affluence de personnes tout en leur garantissant toujours une expérience agréable. Cela permet aussi d'adapter ses recommandations en conséquence, notamment en suggérant de contenus similaires à "BlackList" pendant cette période de l'année où l'audience semble particulièrement réceptive à ce type de contenu.

Graphique 3 : Diagrammes en barre horizontale des saisons d'oeuvres qui vont être bientôt supprimé de la plateforme ainsi que de la durée de visionnage de ces derniers durant l'année 2024



La première partie de la visualisation fut réalisée avec la requête 4. Elle représente les saisons qui vont quitter la plateforme dans deux ans à cause de leur contrat qui arrive à leur fin. Nous avons rajouté la deuxième partie de la visualisation avec l'information de la durée de visionnage sur l'année 2024 pour voir si la saison en question reste intéressante pour les utilisateurs d'Amazone Prime et donc potentiellement prolonger le contrat avec l'équipe de production.

Ainsi, nous voyons que la saison 8 de Blacklist qui est va être en premier supprimer est peu regardée (48 minutes de visionnage) surtout si l'on compare à une œuvre qui ne sera pas bientôt supprimée comme Black Panther avec 139 minutes de visionnage. Cette visualisation permet de prévoir les saisons ou films qui seront supprimés, de l'anticiper pour

ainsi décidé s' il est pertinent pour l'équipe de renouveler leur contrat pour le rendre plus longtemps disponible pour les utilisateurs.

7. Utilisation d'outil d'IA

Dans le cadre de notre projet, nous avons utilisé ChatGPT comme outil d'assistance à plusieurs niveaux. L'outil nous a principalement servi pour générer des données test afin d'alimenter notre base de données, nous aider à formuler des requêtes SQL complexes, améliorer la qualité rédactionnelle de nos textes, déboguer notre code, et mieux comprendre certains concepts techniques liés au projet.

Plusieurs avantages significatifs ont été observés : un gain de temps considérable sur les tâches répétitives, une amélioration de la qualité des textes produits, une efficacité accrue dans l'identification et la résolution des erreurs de code, et une diversité intéressante dans les solutions proposées pour nos requêtes SQL.

Cependant, nous avons également rencontré certaines limites : l'IA a parfois eu du mal à comprendre précisément nos demandes spécifiques, ses suggestions ont systématiquement nécessité une vérification humaine, et nous avons souvent dû adapter les solutions génériques proposées à notre contexte particulier.

En conclusion, l'IA nous a servi d'assistant efficace tout en maintenant notre contrôle sur les décisions importantes du projet, nous permettant ainsi de bénéficier de ses avantages tout en préservant notre maîtrise technique et notre autonomie décisionnelle.

8. Bonus

Nous avons décidé de faire la partie bonus consistant à ajouter des données provenant d'une approche NoSQL. Nous avons décidé d'utiliser le format .json et de passer par le site MongoDB pour créer les nouvelles données.

La logique de sélection des données :

Nous avons décidé d'ajouter des données pour notre classe historique. L'ajout de données est pertinent pour cette classe afin d'avoir une meilleure représentativité d'une historique pour une base de données de plateforme de streaming et de plus elle améliore le rendu de la visualisation où l'on montre le nombre de visionnage en fonction du mois.

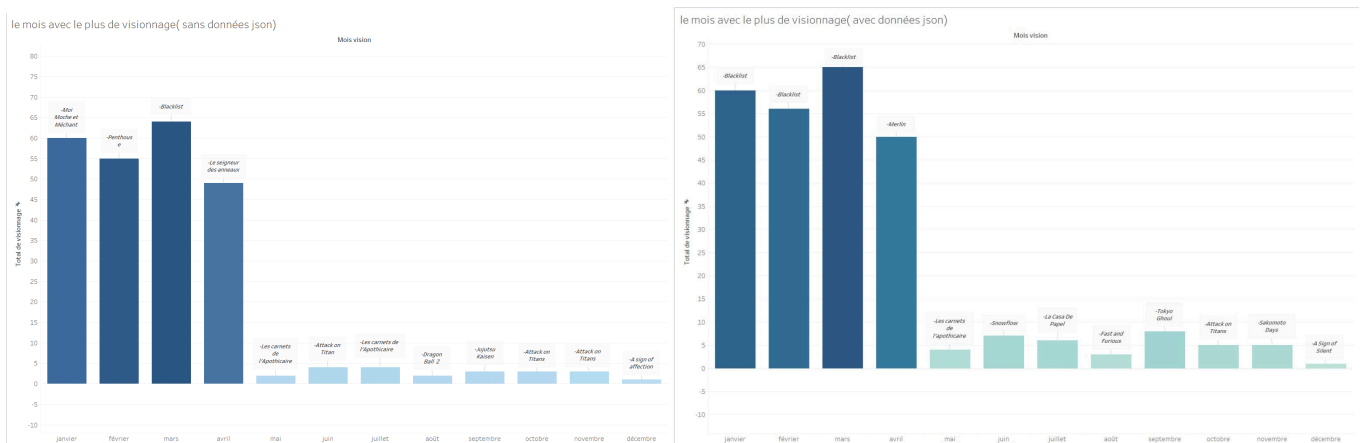
Les étapes de l'ETL suivi :

- **Extraction** : on a créé des données en format .json grâce à MongoDB. On s'est assuré que toutes les données soient différentes et ne se répètent pas avec les données mis avec les données excel.

- **Transformation** : on a récupéré le fichier .json et grâce à un convertisseur en ligne⁵ permettant de passer du format .json au format .csv. Il n'avait pas d'incohérence dans la structure des données.
- **Chargement** : Ensuite, on est partie dans la table Historique qui contenait les données déjà générées en format SQL et on y a importé notre fichier .csv. Pour cela marche, on a changé le séparateur en “,” plutôt qu’en “;” comme pour les autres exportations.

Comparaison des différences observées entre les 2 visualisations (à droite sans les données .json à gauche et avec à droite) :

Graphique 4 : Visualisations de diagrammes en barre verticale du nombre de visionnage de vidéos par mois avec la présentation de l'œuvre la plus visionnée chaque mois



La première différence entre ces visualisations est tout d'abord le nombre de données. Celle sans .json a moins de données que celle avec .json. Les mois avec les plus hauts nombres de visionnage restent les mêmes tout comme ceux avec le moins sûrement car seulement 20 données ont été rajoutées ce qui n'est pas assez pour changer les tendances. Cependant, l'œuvre la plus vue sur les différents mois change. Comme par exemple en Janvier c'est *Moi Moche et Méchant* pour la visualisation dans .json et dans l'autre cas c'est *Blacklist*. Pour d'autres mois on a respectivement pour avril "Les seigneurs des anneaux" et "Merlin", septembre "Jujutsu Kaisen" et "Tokyo Ghoul"... D'autres restent les mêmes comme en Octobre "Attack on Titans".

9. Répartition du travail

La réalisation de ce projet a été répartie de manière équitable entre les membres du groupe, chacun ayant contribué à une partie spécifique en fonction de ses compétences et disponibilités :

- **Base de données (modélisation et alimentation) :**
KPETO Ilem Nancy, CHOUIKHA Samar, et NIAMKE Marie Esther se sont occupées

⁵ Convert JSON to CSV <https://data.page/json/csv>

de la conception et de l'alimentation de la base de données, en créant les différentes classes, en structurant les données dans Excel, puis en les intégrant dans phpMyAdmin.

- **Diagramme de classes et diagramme rationnel :**

Ce travail a été réalisé de manière collaborative par l'ensemble du groupe soit KPETO Ilem Nancy, CHOUIKHA Samar, et NIAMKE Marie Esther NIAMKE Sephora Emmanuela , AYED Mohamed Aziz .Chaque membre a proposé et ajouté les classes qui lui semblaient pertinentes pour représenter les entités et répondre aux besoins fonctionnels.

- **Requêtes SQL :**

Chacun des cinq membres du groupe a réfléchi à une requête SQL spécifique, en lien avec les fonctionnalités de la plateforme. Cela a permis de couvrir différents aspects : abonnements, préférences, contenus exclusifs et réfléchir à comment ils pourraient nous être utiles. Marie Esther Niamke à confectionner les requêtes.

- **Visualisations :**

Les visualisations de données ont été conçues et réalisées par NIAMKE Sephora Emmanuela, KPETO Ilem Nancy, CHOUIKHA Samar sur tableau ,en s'appuyant sur les résultats des requêtes SQL.

- **Rédaction du rapport :**

Le rapport final a été rédigé conjointement par NIAMKE Sephora Emmanuela et AYED Mohamed Aziz, avec l'appui des autres membres pour les relectures et corrections.

10. Conclusion

Ce projet a été particulièrement exigeant en termes de temps et d'organisation. Dès le début du semestre, nous avons dû organiser plusieurs réunions en dehors des cours pour concevoir ensemble la base de données, en tenant compte de la complexité d'un site de streaming, qui nécessite de modéliser un grand nombre de classes pour représenter les multiples fonctionnalités (profils, langues, exclusivités, restrictions, etc.).

La mise en place des tables de données fictives a été facilitée grâce à l'utilisation d'outils d'intelligence artificielle comme ChatGPT, qui nous a permis de générer rapidement une base initiale. Cependant, comme ces outils ne sont pas parfaits, nous avons dû effectuer de nombreux ajustements manuels pour assurer la cohérence et la précision des données.

L'un des aspects les plus nouveaux et difficiles pour nous a été de trouver des requêtes SQL à la fois pertinentes, complexes et liées à la problématique de notre projet. Cela nous a demandé de bien comprendre les relations entre nos tables et de réfléchir à la manière d'extraire des informations utiles pour un service comme Prime Video.

En revanche, la partie sur Tableau Desktop a été plus accessible pour le groupe, car nous avons déjà utilisé cet outil lors du semestre précédent, dans le cadre du cours de visualisation de données. La difficulté s'est située surtout au niveau de l'importation des données depuis MySQL dans Tableau, qui était une étape nouvelle pour nous. Il a fallu comprendre le lien entre les bases externes et l'outil de visualisation, mais nous avons réussi à adapter nos données pour les utiliser efficacement.

Globalement, ce projet nous a permis de renforcer notre compréhension des bases de données relationnelles, de la modélisation UML, des requêtes SQL complexes et de la visualisation, tout en mettant en pratique une vraie démarche collaborative.

11. Sources

Data Page. (n.d.). *JSON to CSV converter*. <https://data.page/json/csv>

GeeksforGeeks. (n.d.). *How to design a database for Amazon Prime*.
<https://www.geeksforgeeks.org/how-to-design-a-database-for-amazon-prime/>

Njihia, I. M. (2023, février 12). *Analyzing Amazon Prime Video database for insights*. Medium.
<https://medium.com/@IsaacM.Njihia/analyzing-amazon-prime-video-database-for-insights-114de020e6d4>

Octopus Team. (n.d.). *Full Amazon Prime dataset*. Kaggle.
<https://www.kaggle.com/datasets/octopusteam/full-amazon-prime-dataset>

Phonandroid. (2023, avril 14). *Prix des abonnements Netflix dans 40 pays du monde*.
<https://www.phonandroid.com/prix-des-abonnements-netflix-dans-40-pays-du-monde.html>

-lien de conversation avec ChatGPT
<https://chatgpt.com/share/6827587f-0a90-8007-ab5d-54fe193650dc>
<https://chatgpt.com/share/682757b8-fac0-8007-8811-27b4d32d9b44>

12. Annexes

12. 1 Annexe A :

-- Requête 1: TOP 5 DES GENRES DES OEUVRES LES PLUS REGARDÉS PAR PAYS.

```
SELECT
    t.nom_pays,
    t.genres_ou_statut
FROM (
    SELECT
        p.nom_pays,
        COALESCE (
            -- on ne prend que les 5 premiers genres
            SUBSTRING_INDEX (
                GROUP_CONCAT (DISTINCT g.genre
```



```

        ORDER BY stats.nombre DESC
        SEPARATOR ', '
    ),
    ', ',
    5
),
    'aucune œuvre visionnée dans ce pays'
) AS genres_ou_statut
FROM pays AS p
LEFT JOIN profil AS pf    ON pf.id_pays    = p.id_pays
LEFT JOIN historique AS h ON h.id_profil    = pf.id_profil
LEFT JOIN video AS v      ON v.id_video    = h.id_video
LEFT JOIN saison AS s     ON s.id_saison    = v.id_saison
LEFT JOIN oeuvre AS o     ON o.id_oeuvre    = s.id_oeuvre
LEFT JOIN genre_oeuvre AS go ON go.id_oeuvre = o.id_oeuvre
LEFT JOIN genre AS g      ON g.id_genre     = go.id_genre

LEFT JOIN (
    SELECT
        pf2.id_pays,
        g2.id_genre,
        COUNT(*) AS nombre
    FROM historique h2
    JOIN profil pf2      ON h2.id_profil    = pf2.id_profil
    JOIN video v2        ON h2.id_video     = v2.id_video
    JOIN saison s2       ON v2.id_saison    = s2.id_saison
    JOIN oeuvre o2       ON s2.id_oeuvre    = o2.id_oeuvre
    JOIN genre_oeuvre go2 ON go2.id_oeuvre  = o2.id_oeuvre
    JOIN genre g2        ON g2.id_genre     = go2.id_genre
    GROUP BY pf2.id_pays, g2.id_genre
) AS stats
    ON stats.id_pays = p.id_pays
    AND stats.id_genre = g.id_genre

GROUP BY p.nom_pays
) AS t
ORDER BY
    (t.genres_ou_statut = 'aucune œuvre visionnée dans ce pays') ASC,
    t.nom_pays ASC;

```

-- Requête 2: NOMBRE DE VISIONNAGES PAR MOIS - pour pouvoir savoir si le nombre de visionnage a augmenter ou diminuer au fil du temps. AVOIR LE MOIS OU ON REGARDE LE PLUS DE D'OEUVRES (le pic)

```

SELECT
    mois_complet.mois AS mois,
    IFNULL(GROUP_CONCAT(v.nom_video ORDER BY v.nom_video SEPARATOR ', '), 'aucune
vidéo visionnée') AS videos_plus_vues,
    IFNULL(MAX(visionnage.total_visio), 0) AS nombre_visionnages
FROM (
    SELECT 1 AS mois_num, 'janvier' AS mois UNION
    SELECT 2, 'février' UNION
    SELECT 3, 'mars' UNION
    SELECT 4, 'avril' UNION
    SELECT 5, 'mai' UNION
    SELECT 6, 'juin' UNION
    SELECT 7, 'juillet' UNION
    SELECT 8, 'août' UNION
    SELECT 9, 'septembre' UNION
    SELECT 10, 'octobre' UNION
    SELECT 11, 'novembre' UNION
    SELECT 12, 'décembre'
) AS mois_complet

LEFT JOIN (
    SELECT
        MONTH(h.date_vision) AS mois_num,
        h.id_video,
        COUNT(*) AS total_visio
    FROM historique h
    GROUP BY mois_num, h.id_video
) AS visionnage ON visionnage.mois_num = mois_complet.mois_num

LEFT JOIN (
    SELECT
        MONTH(h.date_vision) AS mois_num,
        h.id_video,
        COUNT(*) AS total_visio
    FROM historique h
    GROUP BY mois_num, h.id_video
) AS v2 ON v2.mois_num = mois_complet.mois_num AND v2.total_visio = (
    SELECT MAX(v3.total_visio)
    FROM (
        SELECT

```

```

        MONTH(h3.date_vision) AS mois_num,
        h3.id_video,
        COUNT(*) AS total_visio
    FROM historique h3
    GROUP BY mois_num, h3.id_video
) AS v3
WHERE v3.mois_num = mois_complet.mois_num
)

LEFT JOIN video v ON v.id_video = v2.id_video
GROUP BY mois_complet.mois_num, mois_complet.mois
ORDER BY mois_complet.mois_num;

```

-- Requête 3: RECOMMANDATION D'ŒUVRE AVEC LES FAVORIS ET LES LANGUES DOUBLÉES

```

SELECT
    o.nom_oeuvre,
    COALESCE(fav.nombre_favoris, 0) AS nombre_favoris,
    COALESCE(doubl.nombre_langues_doublees, 0) AS nombre_langues_doublees
FROM oeuvre o

```

```

-- sous-requête pour le nombre de favoris
LEFT JOIN (
    SELECT
        id_oeuvre,
        COUNT(DISTINCT id_profil) AS nombre_favoris
    FROM favoris
    GROUP BY id_oeuvre
) AS fav
ON fav.id_oeuvre = o.id_oeuvre

```

```

-- sous-requête pour le nombre de langues doublées
LEFT JOIN (
    SELECT
        o2.id_oeuvre,
        COUNT(DISTINCT ld.id_langue) AS nombre_langues_doublees
    FROM oeuvre o2
    JOIN saison s ON s.id_oeuvre = o2.id_oeuvre
    JOIN video v ON v.id_saison = s.id_saison
    JOIN langue_doublee ld ON ld.id_video = v.id_video
    GROUP BY o2.id_oeuvre
)

```

```

) AS doubl
ON doubl.id_oeuvre = o.id_oeuvre

ORDER BY nombre_favoris DESC, nombre_langues_doublees DESC;

```

-- Requête 4: Requête 4 – Œuvres dont la saison disparaîtra entre le 1er mai de cette année et le 1er mai dans deux ans

```

SELECT
    s.id_oeuvre,
    s.nom_saison,
    s.date_fin_disponibilite,
    DATEDIFF(s.date_fin_disponibilite, CURDATE()) AS jours_restants
FROM saison s
WHERE s.date_fin_disponibilite BETWEEN
    MAKEDATE(YEAR(CURDATE()), 1) + INTERVAL 120 DAY
    AND MAKEDATE(YEAR(CURDATE()), 1) + INTERVAL 2 YEAR + INTERVAL 120 DAY
ORDER BY s.date_fin_disponibilite;

```

-- Requête 5: AFFICHAGE DU PRIX D'ABONNEMENT EN LA DEVISE DE CHAQUE PAYS

```

SELECT
    p.nom_pays AS pays,
    ta.categorie_abonnement AS type_abonnement,

    ROUND(
        prx.montant_chf *
        CASE
            WHEN p.nom_pays = 'Allemagne' THEN 1.02
            WHEN p.nom_pays = 'Australie' THEN 1.50
            WHEN p.nom_pays = 'Belgique' THEN 1.02
            WHEN p.nom_pays = 'Cameroun' THEN 650
            WHEN p.nom_pays = 'Canada' THEN 1.20
            WHEN p.nom_pays = 'Chine' THEN 8.00
            WHEN p.nom_pays = 'Congo' THEN 650
            WHEN p.nom_pays = 'Egypte' THEN 50.00
            WHEN p.nom_pays = 'Etat-Unies' THEN 1.10
            WHEN p.nom_pays = 'France' THEN 1.02
            WHEN p.nom_pays = 'Japon' THEN 160.00
            WHEN p.nom_pays = 'Maroc' THEN 11.00
            WHEN p.nom_pays = 'Mexique' THEN 18.00

```

```

        WHEN p.nom_pays = 'Suisse' THEN 1
        WHEN p.nom_pays = 'Inde' THEN 93.00
        ELSE 1
    END, 2
) AS prix_converti,

CASE
    WHEN p.nom_pays = 'Allemagne' THEN 'EUR'
    WHEN p.nom_pays = 'Australie' THEN 'AUD'
    WHEN p.nom_pays = 'Belgique' THEN 'EUR'
    WHEN p.nom_pays = 'Cameroun' THEN 'XAF'
    WHEN p.nom_pays = 'Canada' THEN 'CAD'
    WHEN p.nom_pays = 'Chine' THEN 'CNY'
    WHEN p.nom_pays = 'Congo' THEN 'XAF'
    WHEN p.nom_pays = 'Egypte' THEN 'EGP'
    WHEN p.nom_pays = 'Etat-Unies' THEN 'USD'
    WHEN p.nom_pays = 'France' THEN 'EUR'
    WHEN p.nom_pays = 'Japon' THEN 'JPY'
    WHEN p.nom_pays = 'Maroc' THEN 'MAD'
    WHEN p.nom_pays = 'Mexique' THEN 'MXN'
    WHEN p.nom_pays = 'Suisse' THEN 'CHF'
    WHEN p.nom_pays = 'Inde' THEN 'INR'
    ELSE '?'
END AS devise

FROM prix prx
JOIN pays p ON p.id_pays = prx.id_pays
JOIN type_abonnement ta ON ta.id_type_abonnement = prx.id_type_abonnement
ORDER BY p.nom_pays, ta.categorie_abonnement;

```

```

-- Requête 6: TRIGGER - AJOUT D'ABONNEMENT CONDITIONNÉ DÛ À UN AUTRE ULTÉRIEUREMENT
ACTIF
DELIMITER $$

```

```

CREATE TRIGGER check_abonnement_condition
BEFORE INSERT ON abonnement
FOR EACH ROW
BEGIN
    DECLARE nb_abos_actifs INT;

    SELECT COUNT(*) INTO nb_abos_actifs
    FROM abonnement

```

```

WHERE id_compte = NEW.id_compte
AND id_actif = 1;

IF nb_abos_actifs > 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = '✗ Ce compte a déjà un abonnement actif.';
END IF;
END$$

DELIMITER ;

```

12. 2 Annexe B :

Dictionnaire de données:

📊 Données pour le projet de Base de données

	A	B	C	D	E	F
3	id_video	int	4	identifiant de la vidéo	video	
4	nom_video	string	100	nom de la vidéo	video	grande taille prévue car le nom peut être codifié (ex:"S01E01VoyageInattendu" pour le nom de la vidéo correspondant à l'épisc
5	duree	int	4	durée de la vidéo en minutes	video	
6	descriptif	string	200	description de la vidéo (résumé)	video	
7	lien_video	string	100	lien d'accès de la vidéo	video	
8	id_saison	int	4	identifiant de la saison	saison	
9	nom_saison	string	50	nom de la saison	saison	ex: Harry Potter a 7 saisons : 1:HP à l'école des sorciers, ...,7: HP et les reliques de la mort (mais 2 vidéos: films "partie I" et "pa
10	date_debut_disponibilite	date	aaaa/mm/jj	date de debut de disponibilité de la saison sur la plateforme	saison	
11	date_fin_disponibilite	date	aaaa/mm/jj	date de fin de disponibilité de la saison sur la plateforme	saison	certaines contenus sont à durée limitées (contrat de diffusion)
12	date_vision	date	aaaa/mm/jj	date de visionnage d'une vidéo par profil	historique	dernière date de visionnage de la vidéo connue.
13	temps_vision	int	10000	temps de visionnage d'une vidéo par profil	historique	10000 car sur une même vidéo on pourrait éventuellement atteindre 100h de visionnage (ex: Avatar vu plusieurs fois)
14	id_profil	int	11	identifiant du profil de l'utilisateur	profil	max 99 999 999 milliards de profils - La table favoris associe chaque profil aux œuvres qu'il a ajoutées en favoris, via une r
15	image_profil	string	100 (URL)	image du profil (choix disponible sur la plateforme)	profil	
16	nom_profil	string	50	pseudonyme du profil	profil	
17	id_restriction	int	1	identifiant de la restriction légale d'âge	restriction	(0:8+, 1:10+, 2:12+, 3:14+, 4:16+, 5:18+)
18	age_restriction	string	10	restriction légale d'âge pour visionner la vidéo	restriction	(0:8+, 1:10+, 2:12+, 3:14+, 4:16+, 5:18+)
19	id_abonnement	int	10	identifiant de l'abonnement	abonnement	un abonnement peut être mis en veille pendant une certaine période. ID différent car un compte peut avoir plusieurs abonne
20	date_debut	date	aaaa/mm/jj	date de début de l'abonnement	abonnement	de là avec le type d'abonnement on déduit la date de fin
21	id_compte	int	10	identifiant du compte utilisateur	compte	
22	nom_utilisateur	string	100	nom de famille de l'utilisateur	compte	plus long du monde: 102 lettres
23	prenom_utilisateur	string	100	prénom de l'utilisateur	compte	plus long du monde: 102 lettres
24	email	string	50	adresse mail de l'utilisateur	compte	
25	date_naissance	date	aaaa/mm/jj	date de naissance de l'utilisateur	compte	de là on en déduit l'âge pour conclure par la restriction d'âge légale à appliquer au contenu visible
26	mot_de_passe	string	50	mot de passe de l'utilisateur	compte	max 50 caractère pour mot de passe (personne ne mémorise davantage)
27	id_oeuvre	int	10	identifiant de l'oeuvre (vidéos regroupées sous une oeuvre globale)	oeuvre	max: 9 999 999 999 (mais moins que saisons car films d'une trilogie correspondront à 3 saisons mais 1 seule oeuvre
28	nom_oeuvre	string	50	nom de l'oeuvre générale	oeuvre	ex: l'oeuvre Harry Potter regroupe 8 vidéos de type film sous 7 saisons différentes (1 "saison" par film pour ordonner la chronol
29	id_categorie	int	1	identifiant du genre	genre	max 9 catégories: film, series, anime, film animation, documentaire, film3D
30	categorie	string	50	genre affilié à l'oeuvre	genre	ex:Horreur, Romance, Action, SF, Seinen,...
31	id_langue	int	5	identifiant	langue	selon des études, il y aurait environ 10 000 langues parlées dans le monde. Présence de la table Associative langue doublée: c
32	nom_langue	string	30	langue originale de la vidéo	langue	
33	id_person	int	10	identifiant des personnes intervenantes dans la vidéo	intervenant	max: 9 999 999 999 milliard d'employé possible (notons que certains employé seront morts mais les films perdurent !)
34	nom	string	50	nom des personnes intervenantes	intervenant	

RÈGLES GESTION : ABONNEMENT -- Nous ne pouvons pas ajouter un nouvel abonnement à un compte tant qu'il y a un déjà actif.

RÈGLES GESTION : SAISON -- Les œuvres ayant des SAISON 0 sont considérées comme étant des films ou des animés n'ayant pas de saison. Exemple : Tel que ONE PIECE qui à 0 saison et environ 1000 épisodes.